

Pauta de Evaluación Sumativa 4: Desarrollo de Aplicaciones Front End con React y Persistencia Local

1. Cuadro de Identificación Técnica de la Evaluación

Esta evaluación constituye un pilar estratégico en la formación del Programador Front End, marcando la transición técnica desde interfaces puramente visuales hacia aplicaciones robustas con estado persistente y lógica de negocio. En un ecosistema profesional, el manejo de datos asíncronos y la sincronización con el almacenamiento local no son opcionales, sino requisitos para construir soluciones resilientes. El estudiante deberá demostrar su capacidad para orquestar hooks de React, manipular el DOM virtual de forma eficiente y garantizar la integridad de la información frente a fallos externos o recargas de página.

Parámetro Técnico	Detalle
Ponderación	25%
Puntaje Máximo	21 puntos
Exigencia	60%
Puntaje de Corte (Nota 4.0)	13 puntos

2. Contexto de la Problemática (ABP): 'Cocina Chilena'

Usted ha sido contratado como Desarrollador Front End por '**Cocina Chilena**', una cadena de comida que desea digitalizar su menú físico para mejorar la agilidad de sus operaciones. La empresa requiere un sistema unificado para administrar platos típicos chilenos, permitiendo a los administradores locales gestionar precios dinámicos y la disponibilidad de stock en tiempo real. La solución debe ser una Single Page Application (SPA) que consuma datos externos y permita la persistencia de cambios de forma local, evitando que la información se pierda al cerrar el navegador.

Capa de Valor Analítico: Beneficios de la Digitalización

La transformación de un menú estático a uno dinámico en React ofrece escalabilidad inmediata: permite inyectar nuevos campos de datos (como alérgenos o promociones) sin reestructurar la interfaz. Desde la perspectiva de la Experiencia de Usuario (UX), la reactividad del estado asegura que el administrador reciba retroalimentación instantánea, mientras que el uso de estados de carga (loading) previene la incertidumbre durante la latencia de red, elevando el estándar profesional del producto.

3. Arquitectura Modular y Archivos a Crear

La arquitectura de este proyecto se basa en el **Principio de Responsabilidad Única (SRP)**. No se aceptarán implementaciones monolíticas; el código debe estar segmentado de forma atómica para facilitar su depuración y escalabilidad.

Se exige la creación de los siguientes archivos en el directorio `src/`:

1. **`services/api.js`**: Su única responsabilidad es gestionar la comunicación externa. Debe usar el patrón `fetch` para retornar una **Promesa** con los datos brutos. Los componentes no deben llamar a `fetch` directamente.
2. **`App.jsx`**: Actúa como el orquestador principal (Source of Truth). Gestiona el estado global, la hidratación de datos y la sincronización con `localStorage`.
3. **`App.css`**: Contiene la capa de estilos globales. Es obligatorio el uso de un **Reset Global** mediante el selector universal `* { box-sizing: border-box; }` para asegurar que rellenos y bordes no alteren el tamaño calculado de los elementos.
4. **`components/Navbar.jsx`**: Componente de navegación funcional.
5. **`components/PlatoCard.jsx`**: Componente de presentación encargado de renderizar la información de un plato individual y emitir eventos de edición/eliminación.
6. **`components/MenuPizarra.jsx`**: Contenedor flexible que organiza las tarjetas. Debe implementar un diseño basado en **Flexbox**, utilizando propiedades como `gap` y `justify-content: space-evenly` para una distribución armónica.

4. Desafío de Investigación y Transformación de Datos

El estudiante debe investigar la documentación de **TheMealDB** y consumir exclusivamente el endpoint de platos nacionales: (pista: `filter.php?a=Chile.`)

Capa de Valor Analítico: Hidratación y Eficiencia

El filtrado del lado de la API es superior al del cliente porque reduce drásticamente el ancho de banda y la carga computacional del navegador. No obstante, los datos de la API (`idMeal`, `strMeal`, `strMealThumb`) son insuficientes para 'Cocina Chilena'.

Requisito de Arquitectura: Tras el fetch, el estudiante debe realizar una **Transformación de Datos (Mapping)**. Cada objeto recibido debe ser "hidratado" con campos adicionales por defecto: `precio: 0` y `disponible: true`. Esta mutación controlada prepara el objeto para la lógica de negocio local.

5. Flujo Detallado del CRUD y Persistencia Local

5.1 Estado Inicial Perezoso (Lazy Initializer) con Control de Errores

El estado `menu` en `App.jsx` debe inicializarse evitando lecturas redundantes. Es imperativo envolver la lectura de `localStorage` en un bloque `try-catch` para prevenir fallos catastróficos si el JSON almacenado está malformado:

```
const [menu, setMenu] = useState(() => {
  try {
    const saved = localStorage.getItem('cocina_chilena_menu');
    return saved ? JSON.parse(saved) : [];
  } catch (error) {
    console.error("Error cargando persistencia:", error);
    return [];
  }
});
```

5.2 Sincronización con `useEffect`

Implemente un efecto que ejecute `localStorage.setItem` de forma automatizada cada vez que el estado `menu` sufra una modificación, garantizando la persistencia reactiva.

5.3 Operación CREATE y Validación Defensiva

Al añadir un plato al menú, se debe capturar el precio. Se exige el uso de la función `Number()` para la conversión y la validación mediante `Number.isNaN()`. El sistema debe rechazar cualquier entrada que sea `NaN` o valores negativos, protegiendo la integridad financiera de la aplicación.

5.4 Operación READ (Renderizado Eficiente)

Utilice el método `.map()` para transformar el estado en elementos visuales. Es obligatorio el uso de la propiedad `key` basada en el `idMeal` para optimizar el proceso de reconciliación de React.

5.5 Operación UPDATE Inmutable

Para la edición de precios, se debe emplear `.map()` para retornar un nuevo arreglo, respetando la inmutabilidad. Aplique las mismas validaciones de `Number.isNaN()` que en la creación. El impacto de la inmutabilidad es crítico: asegura que React detecte el cambio de referencia y re-renderice solo lo necesario.

5.6 Operación DELETE

Utilice el método `.filter()` para eliminar platos. Esta operación debe generar un nuevo estado sin mutar el arreglo original.

5.7 React Defensivo: Carga y Red

Gestione los estados de `loading` (mientras la Promesa de `api.js` se resuelve) y `error` (fallos de red). El componente debe ofrecer un botón de "Reintentar" que dispare nuevamente el flujo asíncrono.

6. Matriz de Rúbrica Oficial (21 puntos)

Criterio	Excelente (3 pts)	Bueno (2 pts)	Regular (1 pt)	Deficiente (0 pts)
API y Transformación	Consume API asíncronamente en archivo separado; aplica mapping (precio/disponible) y maneja errores.	Consume la API pero no realiza la transformación de datos (mapping) requerida.	Consumo de API realizado directamente en el componente UI.	No consume la API o usa endpoint incorrecto.
Arquitectura Modular	6 archivos según SRP; <code>api.js</code> retorna Promesas; configuración impecable en Vite.	Estructura completa pero con lógica de API mezclada en componentes.	Faltan archivos clave o la modularidad es inconsistente.	Estructura monolítica.
LocalStorage Seguro	Implementa Lazy Initializer con <code>try-catch</code> y sincronización mediante <code>useEffect</code> .	Usa <code>localStorage</code> pero sin Lazy Init o sin manejo de excepciones.	Persistencia básica que falla si el storage está vacío o corrupto.	No implementa persistencia.
CREATE y Validación	Validación estricta usando <code>Number()</code> y <code>Number.isNaN()</code> para evitar precios inválidos.	Agrega elementos pero con validaciones débiles (permite negativos o NaN).	Agrega elementos sin validar ningún tipo de dato.	No implementa creación.

Inmutabilidad (U/D)	Actualización y eliminación realizadas exclusivamente con <code>.map()</code> y <code>.filter()</code> sin mutación.	Operaciones funcionales pero con rastros de mutación directa al estado.	Solo implementa una de las dos operaciones (Update o Delete).	No permite editar ni eliminar.
Diseño con Flexbox	Layout con <code>gap</code> y <code>justify-content:</code> <code>space-evenly;</code> uso de <code>box-sizing: border-box.</code>	Diseño funcional con Flexbox pero carece de reset global o espaciado uniforme.	Diseño básico con fallos de alineación o desborde de elementos.	No aplica Flexbox.
Ética e Integridad	Declaración completa de IA con prompts y justificación técnica en formato APA.	Declaración presente pero carece de justificación técnica o formato APA.	Mención vaga al uso de IA sin anexo de prompts.	No incluye declaración ética ni anexo.

8. Formato de Entrega y Anexo de IA

Anexo de Inteligencia Artificial (Obligatorio)

Debe documentar el uso de IA para indicar el caso de uso, la razon del requerimiento y un resumen de lo respondido por la IA. **Recuerda que debes adjuntar este archivo en la entrega, si no se considerará el puntaje mínimo en el ítem correspondiente.**

Prompt utilizado	¿Por qué lo pediste?	Resumen de respuesta